

SortBench performance analysis

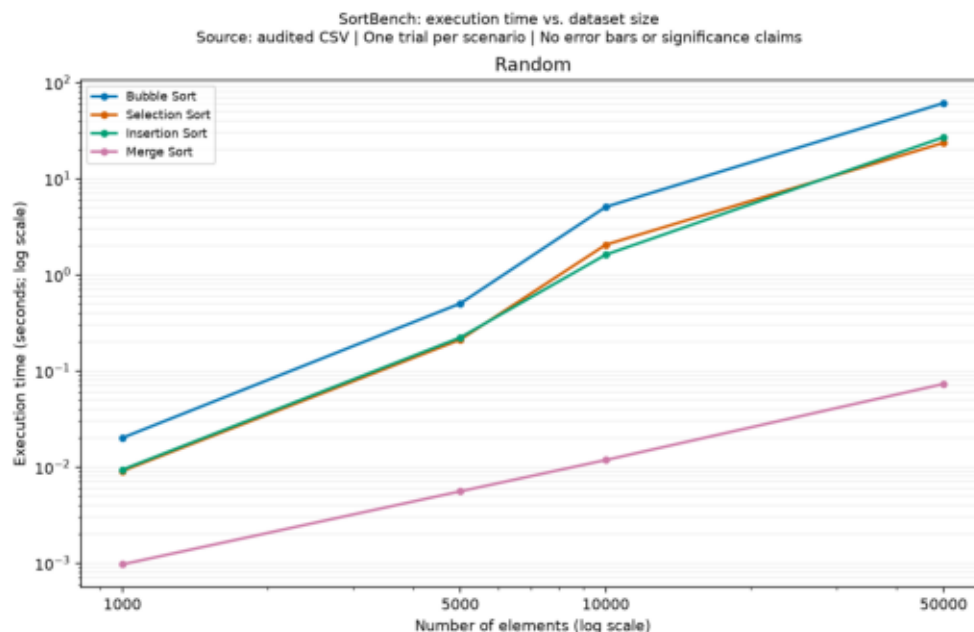
Final analysis for SortBench v1.0.0. A formatted three-page version is available in PERFORMANCE_ANALYSIS.pdf. All times below come from the primary CSV, unless labeled diagnostic.

Experiment and evidence

SortBench examines how input size and ordering affect four manually implemented sorting algorithms: Bubble, Selection, Insertion, and Merge Sort. The experiment contains 64 scenarios: four algorithms, four orderings, and sizes of 1,000, 5,000, 10,000, and 50,000 integers. Each scenario has one recorded trial. For each size, all orderings share the same seeded multiset. Random input uses the generated order, sorted and reverse-sorted inputs reorder those values, and partially sorted input begins sorted before applying size // 20 random swaps. This definition does not imply that exactly 5% of values are misplaced.

Measurements used Python 3.14.4 on Windows 11 with the AMD processor identifier preserved in the metadata. Algorithms ran sequentially, each receiving an independent copy. Only the sorting call was timed with time.perf_counter(): generation, copying, correctness checks, and file output were excluded. Each output had to equal Python's independently computed sorted(original) and preserve the supplied list's identity. A separate audit checked all 64 saved rows and their metadata.

An initial run recorded 3,927.57 seconds for reverse-sorted Bubble Sort at 50,000 elements, alongside a large wall-time/CPU-time discrepancy. A long pause is suspected, but its cause is unconfirmed. The entire initial attempt was archived. A complete repeat, with a temporary system-awake request, became the primary dataset; it took 396.21 seconds wall time and 393.31 seconds CPU time. No individual fast rows were selected across attempts. A separate 32-case diagnostic at 5,000 and 10,000 elements checks variability and is not averaged into the primary data.



Algorithm behavior and complexity

The bounds below describe these implementations. Space excludes the input list; stability means preserving the relative order of equal keys.

Algorithm	Best time	Average time	Worst time	Extra space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

Bubble Sort swaps adjacent inversions and stops after a pass without swaps. That optimization makes sorted input linear, but large disordered inputs still require many comparisons and swaps. Selection Sort repeatedly searches the unsorted suffix for its minimum. Its simplicity and at most $n-1$ swaps can be useful when swaps are costly, but the suffix scans remain quadratic even for sorted input; this experiment did not measure a costly-swap workload.

Insertion Sort shifts larger prefix values to insert the next item. It is adaptive: few inversions mean few shifts, while reverse order approaches its quadratic worst case. Merge Sort recursively combines sorted ranges and uses one reusable auxiliary buffer. It offers $O(n \log n)$ time across orderings, at the cost of $O(n)$ extra storage plus $O(\log n)$ recursion depth. All four return the original list, so a shared interface does not imply equal memory requirements. Memory usage was not profiled.

Measured comparisons

The table summarizes 50,000-element results in seconds. The complete comparison covers all sizes; the size chart and ordering chart visualize the same 64 measurements. Logarithmic axes accommodate the wide timing range; each plotted point is an actual observation, not an estimated runtime.

Ordering	Bubble	Selection	Insertion	Merge
Random	61.1791862	23.6448514	27.0068737	0.0731335
Sorted	0.0012729	29.2820558	0.0025953	0.0694641
Reverse-sorted	79.5162674	28.0274392	52.9517419	0.0631464
Partially sorted	40.8928520	26.2688121	3.2474878	0.0819030

Merge Sort had the lowest recorded time in all 12 random, reverse-sorted, and partially sorted size/type groups. It was fastest even at the smallest tested size of 1,000, so these results do not support assuming that Insertion or Selection Sort wins merely because a dataset is called small. On random input, growing from 1,000 to 50,000 values increased Merge's time about 75.5 times, versus approximately 2,606-3,047 times for the quadratic algorithms. For comparison, the corresponding theoretical $n \log n$ growth factor is about 78.3 and the n^2 factor is 2,500. These observations are consistent with the theoretical distinction, but four single-trial points cannot establish an exact asymptotic model or predict every intermediate size.

Ordering effects

Sorted input substantially changed the result. Bubble won at 1,000, 5,000, and 50,000, while Insertion won at 10,000. From 1,000 to 50,000, their times grew about 49.0 and 55.2 times respectively, consistent with their linear best-case behavior. Selection still scanned the suffixes, taking 29.2821 seconds at 50,000, despite the input already being ordered. Merge's general divide-and-merge procedure also did more work than the adaptive algorithms needed for this special case.

Partial order helped Insertion considerably: at 50,000 it took 3.2475 seconds, compared with 27.0069 seconds on random input and 52.9517 on reversed input. Nevertheless, Merge was still fastest on the project's partial-swap datasets. Random swaps can move values long distances and create many inversions; these findings do not establish a universal ranking for every meaning of "nearly sorted."

Recommendations and limitations

For the tested random, reversed, and partially sorted integer datasets, Merge Sort is the measured time-based recommendation when its extra storage is acceptable. For already sorted input, Bubble and Insertion are strong adaptive choices. The recommendation function reports the exact primary winner for each supported scenario, retains exact ties, and rejects missing or untested scenarios. It does not guess a crossover size or extrapolate beyond the four measured sizes. Memory-sensitive applications should also consider the constant-space algorithms; Selection's limited swaps and lack of stability are separate trade-offs, not evidence of measured superiority.

The primary sorted 10,000-element winner was Insertion at 0.0004652 seconds, versus Bubble at 0.0006886. In the separate diagnostic, Bubble instead led by only about 1.2 microseconds. Partial-order Merge timings also fluctuated: the primary 10,000 case was slightly faster than 5,000, while the diagnostic restored increasing times. These examples show why single observations, especially short ones, should not be presented as stable rankings. Fixed execution order, machine load, timer effects, and one seed per size limit generalization. There are no confidence intervals or error bars, and the rejected initial run does not remove all possible noise from the repeat.

Overall, the measurements support Merge's scalability on disordered input and the benefit of adaptivity on already sorted input. Choosing an algorithm requires matching input characteristics, storage constraints, and stability needs. Repeated trials with varied seeds and randomized execution order would strengthen future conclusions; the current study provides transparent, correctness-checked evidence within its stated scope.